

THE ANALYSIS PROCESS STRUCTURED AND OBJECT- ORIENTED MODELING



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

MODULE 4

LEARNING OBJECTIVES

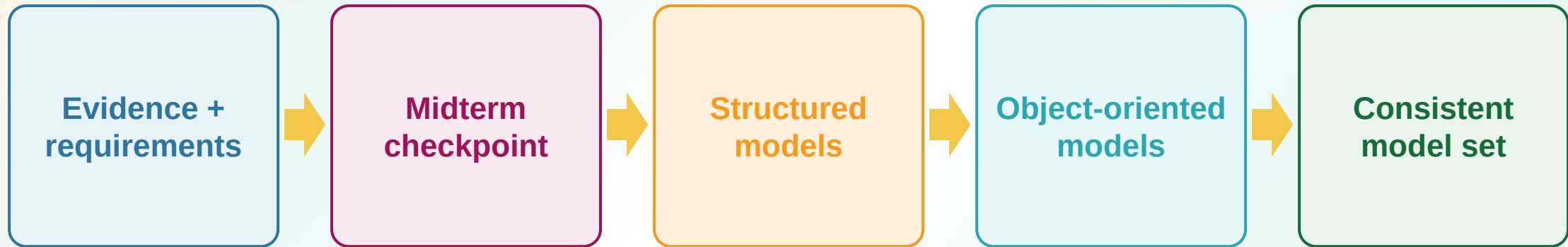
What you will learn

ILO 5: Create, validate, and reconcile structured, object-oriented, process, data, and behavioral analysis models.

CLO alignment: Demonstrate CLO 3 and CLO 4 by producing accurate, consistent, specialization-appropriate, and traceable models for the progressive Title Proposal portfolio.

Module 4 begins with a validated requirements baseline

The midterm covers Modules 1–3; this module converts Module 3 requirements into visible, reviewable analysis models.



Do not model assumptions as facts. Every major process, actor, data object, and rule must trace backward to validated evidence.

Question of understanding

If two diagrams describe the same system but tell different stories, which one should the team trust?



FEU ALABANG



FEU DILIMAN

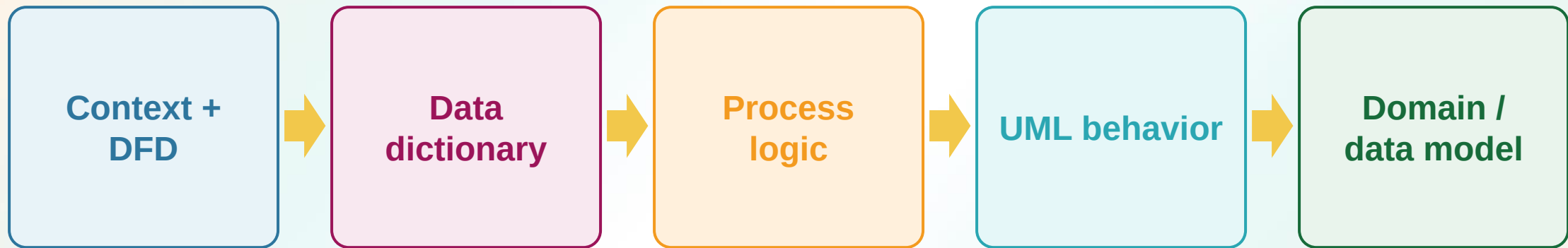


FEU TECH

Technology Driven by Innovation

One system needs several connected views

Each model answers a different question; consistency makes the combined explanation trustworthy.



Requirements and the RTM connect every view to stakeholder evidence, acceptance criteria, and the project scope.

STRUCTURED ANALYSIS AND LOGICAL MODELING



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Logical models focus on required behavior

Separate what information moves and transforms from decisions about software, hardware, database products, screens, or code.

LOGICAL

**What must happen?
What information is needed?
Who or what exchanges it?
Which rules govern it?**

PHYSICAL

**Which technology?
Which screen or service?
Which database or device?
Which deployment choice?**

Module 4 begins with the logical view; solution design follows in later modules.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

A context diagram defines the system as one process

It shows the system boundary, external entities, and the named data flows that cross that boundary.

Include	Purpose	Exclude
One system process	Name the system being analyzed	Internal subprocesses
External entities	Show sources and destinations	Internal data stores
Boundary data flows	Show information crossing scope	Control flow or technology

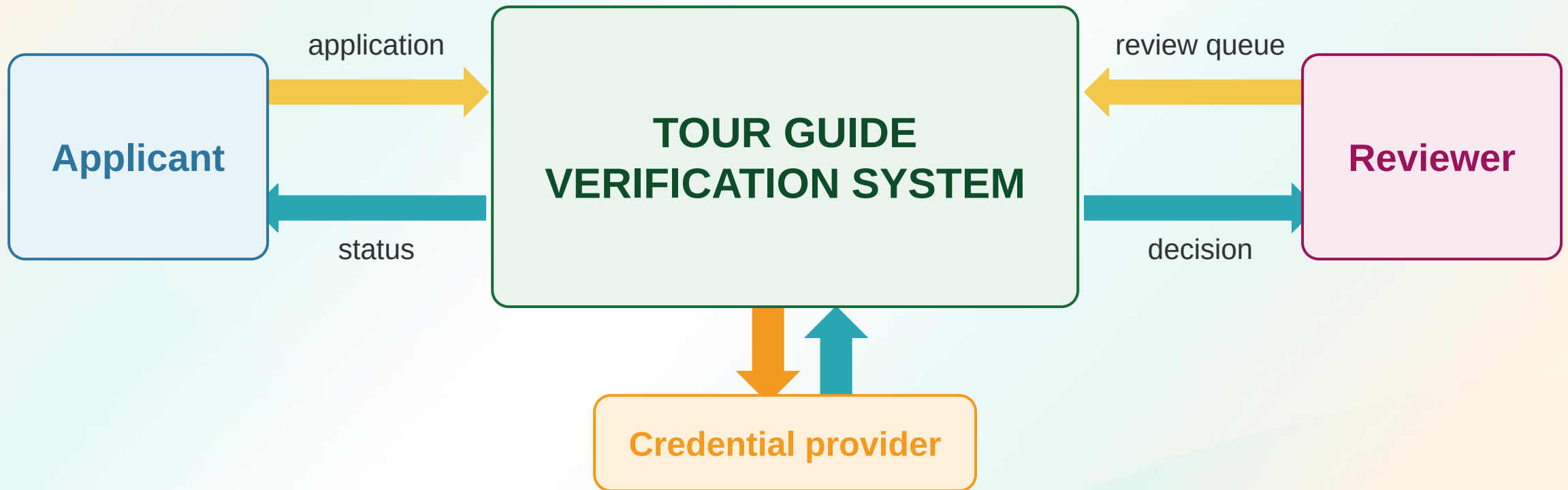
Boundary decisions determine what the project promises

Place an activity inside the system only when the proposed solution is responsible for transforming or storing its information.

Boundary question	Inside when...	Outside when...
Who performs it?	The system executes the behavior	A person or external service acts
Who controls it?	The project owns the logic	Another organization controls it
What is exchanged?	Information is transformed internally	Only an interface contract is known
What is committed?	Included in approved scope	Future or dependency scope

Worked context: Tour Guide Verification System

The system is one process; applicants, reviewers, and the credential provider remain external.



A DFD uses four core symbol types

Choose one notation family and apply it consistently; meaning matters more than decorative variation.

PROCESS

Transforms data
Verb + noun

DATA FLOW

Moves information
Noun phrase

DATA STORE

Retains
information
Plural noun

EXTERNAL ENTITY

Source or
destination
Role /
organization

A DFD models data movement and transformation—not timing, user-interface layout, or program sequence.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

DFD names should explain meaning at a glance

Use verb–noun phrases for processes and precise noun phrases for flows, stores, and external entities.

Element	Weak name	Stronger name
Process	Handle data	Validate credential
Data flow	Information	Credential evidence
Data store	Database	Guide applications
External entity	User	Tourism reviewer



Use one leveling convention throughout the project

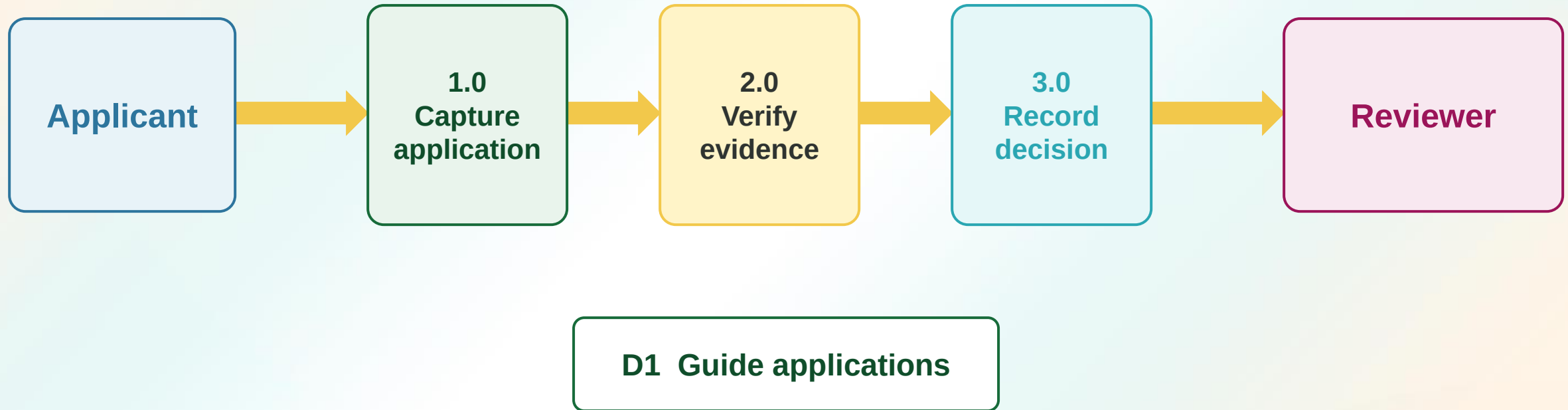
This deck uses Context → Level 0 → Level 1; some references number the first decomposition differently, so label the convention explicitly.



Decompose for understanding—not to produce the largest possible diagram.

Level 0 reveals the major logical processes

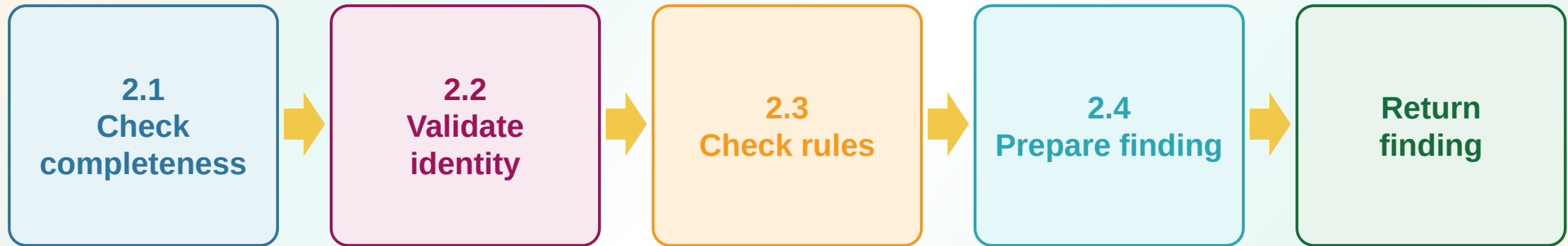
The context process is decomposed while preserving the external flows that cross the system boundary.



This teaching view is simplified; teams must add all validated flows required by their case.

Level 1 preserves the parent contract

Process 2.0 is decomposed into internal steps while its parent inputs and outputs remain visible.



Parent input: credential evidence | Parent output: verification finding |
Internal flows may be added.

Balancing preserves the parent process contract

Every external input and output of the parent process must reappear in the child diagram with the same meaning.

Parent 2.0	Child Level 1	Balanced?
Credential evidence →	Credential evidence → 2.1	Yes
Verification finding →	2.4 → Verification finding	Yes
No alert output	2.3 → Security alert	No: new external flow

Internal child flows do not need to appear on the parent; new boundary flows do.

Balancing lab: find the defect

Compare the parent contract with the proposed child diagram before evaluating the internal logic.

PARENT 3.0

Inputs

- verification finding
- reviewer decision

Output

- recorded status

CHILD 3.x

Inputs

- verification finding

Outputs

- recorded status
- applicant notification

Defects: the reviewer decision disappeared, and applicant notification was introduced without a parent flow.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

A data flow names information—not an action or channel

Define its contents in the data dictionary and confirm that a process creates, transforms, or consumes it.

Check	Weak	Better
Meaning	Send	Verification finding
Contents	Data	Guide ID + rule results + status
Direction	Unclear arrows	Named source and destination
Channel	Email flow	Notification content

Data stores retain information for later use

Name the logical collection and show at least one process that writes or updates it and one that reads it when required.

Rule	Valid example	Defect to avoid
Logical name	Guide applications	MySQL database
Created or updated	1.0 → D1	Entity → D1 directly
Read for purpose	D1 → 2.0	Store with no consumer
Dictionary link	D1 structure defined	Undefined fields



External entities sit outside the system boundary

Name the role, organization, device, or external system that originates or receives information across the boundary.

Question	Strong answer	Weak shortcut
Who or what?	Tourism reviewer	User
Why external?	Human decision remains outside	Because the diagram is full
What crosses?	Decision + rationale	Click
Duplicate roles?	Merge only if flows and goals match	Merge by convenience

Decompose a process when its internal logic hides important work

Expand only enough to expose decisions, transformations, stores, and exception handling required for validation.



Stop when a process can be specified clearly with structured English, a decision table, a decision tree, or a business rule reference.

A primitive process is detailed enough to specify

It has one coherent responsibility, defined inputs and outputs, known rules, and logic that can be reviewed without another DFD level.

Primitive-process test	Ready	Needs decomposition
Responsibility	One transformation	Several unrelated outcomes
Rules	Logic can be written	Rules remain hidden
Data	Inputs and outputs defined	Unknown data content
Traceability	Links to requirements	No supporting requirement



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

DFD rules prevent impossible data movement

Every data flow must involve a process when information is created, transformed, validated, routed, or stored.

Invalid pattern	Why invalid	Correction
Entity → entity	System does no transformation	Insert a responsible process
Store → store	No process controls movement	Add a transfer process
Entity → store	External actor bypasses system logic	Capture through a process
Black hole / miracle	Outputs or inputs lack explanation	Add required flows or rules

Validate a DFD for meaning—not only notation

A technically neat diagram can still omit a stakeholder, exception, rule, data need, or requirement.

BOUNDARY

Does scope
match
the proposal?

COVERAGE

Do all key flows
appear?

BALANCE

Do levels
preserve
contracts?

MEANING

Do names
match
the dictionary?

TRACE

Does each
element
have a reason?

A walkthrough should use real scenarios, including exceptions and unauthorized or invalid conditions.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

The data dictionary is the shared vocabulary behind the models

Define every important element, structure, flow, store, alias, domain, and metadata rule once—then reuse the definition consistently.



If two diagrams use the same term differently, the data dictionary should expose the conflict.

A data element definition must be precise enough to validate

Record its meaning, source, type or format, allowed domain, required status, sensitivity, and related rules.

Field	Example: guide_status	Why it matters
Definition	Current review disposition	Prevents meaning drift
Domain	Draft Submitted Verified Rejected	Supports validation
Owner	Tourism review lead	Assigns authority
Sensitivity	Internal operational	Guides protection

Data structures define flows and stores

Use a readable notation and define optional, repeating, and alternative groups consistently.

Structure	Composition	Interpretation
Guide application	applicant + credentials + contact	Required groups
Credentials	{credential}1..*	One or more repeating items
Contact	email + [mobile]	Mobile is optional
Decision	verified rejected returned	One allowed alternative

Aliases, domains, and metadata resolve hidden ambiguity

Record alternate labels, valid values, lineage, ownership, update frequency, quality rules, sensitivity, and retention where relevant.

Dictionary control	Example	Conflict prevented
Alias	guide ID = accreditation number	Different names for one field
Domain	status $\in$ approved values	Uncontrolled labels
Lineage	provider response $\rightarrow$ finding	Unknown origin
Metadata	owner, timestamp, sensitivity	Unmanaged data

Worked dictionary entries must agree with the DFD

The flow, store, and elements should use the same names and composition across every model and requirement link.

Name	Type	Definition / composition	Trace
Credential evidence	Data flow	guide ID + credential set	R-04
Guide applications	Data store	application + status + history	R-02, R-07
review_decision	Element	Verified Rejected Returned	BR-03
Verification finding	Structure	rule results + rationale + time	R-05

A process specification explains one primitive process

State its purpose, trigger, inputs, outputs, preconditions, postconditions, normal logic, exceptions, and linked rules.

P-SPEC field	Process 2.2 Validate identity
Trigger	Complete credential evidence received
Input / output	Evidence → identity result
Precondition	Applicant and credential IDs are present
Normal logic	Compare identifiers with approved provider response
Exception	Provider unavailable → pending review



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Structured English makes process logic explicit

Use a limited vocabulary, clear indentation, defined data names, and explicit decisions or repetition.

```
IF credential evidence is complete
  IF provider response matches applicant identity
    SET identity result to VALID
  ELSE
    SET identity result to REVIEW REQUIRED
ELSE
  SET identity result to INCOMPLETE
END IF
```

Use dictionary terms exactly; do not hide a new requirement inside the logic.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Decision tables expose combinations that prose often misses

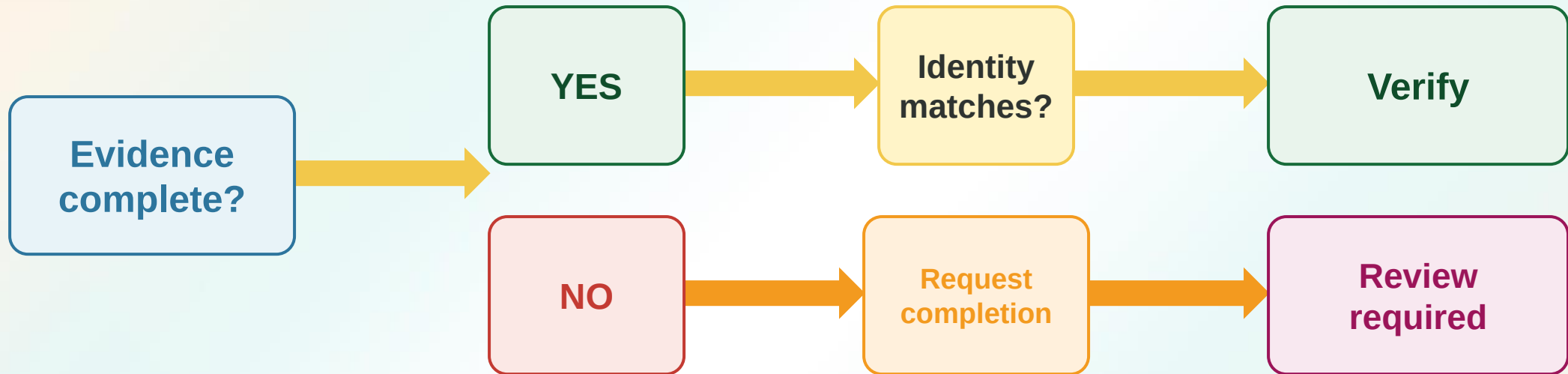
List conditions, enumerate relevant combinations, assign actions, and remove impossible or equivalent rules carefully.

Condition / action	Rule 1	Rule 2	Rule 3	Rule 4
Complete evidence?	Y	Y	N	N
Identity matches?	Y	N	–	–
Provider available?	Y	Y	Y	N
Verify	X			
Review required		X	X	
Pending				X



Decision trees clarify the order of nested decisions

Use them when the sequence of conditions matters and reviewers need to see how one decision leads to the next.



Choose the logic representation that best exposes the rule

Structured English, decision tables, and decision trees are complementary; the primitive process determines the best form.

Use	Best when	Watch for
Structured English	Sequence and limited branching	Ambiguous terms
Decision table	Many condition combinations	Missing or impossible rules
Decision tree	Order of decisions matters	Repeated branches
Rule reference	Authoritative policy already defines logic	Outdated or inaccessible source

Business rules must remain traceable and owned

Each rule needs an identifier, statement, authority, effective scope, owner, related requirements, and affected model logic.

Rule	Statement	Authority	Model link
BR-03	A reviewer records one disposition per review cycle	Review policy	P3.0 / UC-04
BR-05	Rejected evidence includes a reason code	Service standard	P3.2 / AC-07
BR-08	Only assigned reviewers may decide	Access policy	UC-04 / SEC-02

Traceability connects evidence, requirements, and model elements

A model element without a requirement may be scope invention; a requirement without a model link may be missing from the analysis.



Update the RTM with model identifiers so reviewers can move backward and forward through the reasoning.

Defect hunt: find the model that cannot be true

Review one diagram at a time, then compare its names, flows, rules, and scope with the rest of the model set.

Defect	Evidence	Correction
Unbalanced flow	Child adds security alert	Add parent flow or remove invention
Undefined data	Status has no dictionary domain	Define allowed values
Logic mismatch	Table permits action the P-SPEC rejects	Resolve the governing rule
Scope drift	DFD includes payment not in requirements	Remove or approve change

Diagramming tools support discipline—but do not guarantee correctness

Use naming standards, layers, alignment, version labels, comments, export settings, and a review checklist before baselining any model.



A CASE or diagramming tool can draw an invalid model perfectly. Rules and cross-model review remain the analyst's responsibility.

Cybersecurity: model protected flows and decision responsibility

Keep the operational transaction visible while tracing sensitive data, trust crossings, validation findings, and authorized decisions.

Model focus	Example	Quality question
External entity	Reporter / identity provider / responder	Is the role truly outside?
Data flow	Incident evidence / validation result	Is sensitivity defined?
Process	Validate report / authorize response	Who owns the decision?
Store	Evidence record / audit history	Who may read or update?

Web and mobile: model the complete service journey

Represent client input, validation, backend processing, notifications, offline or retry conditions, and external service dependencies logically.

View	Example	Consistency check
Context	Applicant ↔ service platform ↔ reviewer	Boundary matches scope
DFD	Submit → validate → review → notify	Flows balance across levels
Dictionary	submission, status, error response	API and model terms agree
P-SPEC	Retry, validation, and failure rules	Exception behavior is explicit

Animation and game development: model creation and play

Separate asset-production flows, content approval, player events, progression data, safety checks, and operational support.

Process area	Data movement	Rule or exception
Asset production	source asset → reviewed asset	Version and approval required
Content validation	lesson content → finding	Subject expert decision
Player session	input → state update → feedback	Accessibility and safety
Recovery	saved state → restored session	Corrupt or missing save

Business analytics: model data before the dashboard

Trace sources, ingestion, validation, transformation, storage, metrics, access, and decision outputs before choosing visualizations.

Stage	Data artifact	Control question
Source	transaction extract / survey	Owner and definition?
Validate	quality finding	What fails and who resolves it?
Transform	derived measure	Formula and lineage defined?
Publish	metric / report / alert	Who may interpret and act?

OBJECT-ORIENTED ANALYSIS AND UML

Object-oriented analysis organizes the domain around responsibilities

Use UML views to describe actors and goals, workflows, interactions, domain concepts, relationships, and state changes.



UML is a modeling language, not a complete development process. Use only the views that answer the project's analysis questions.

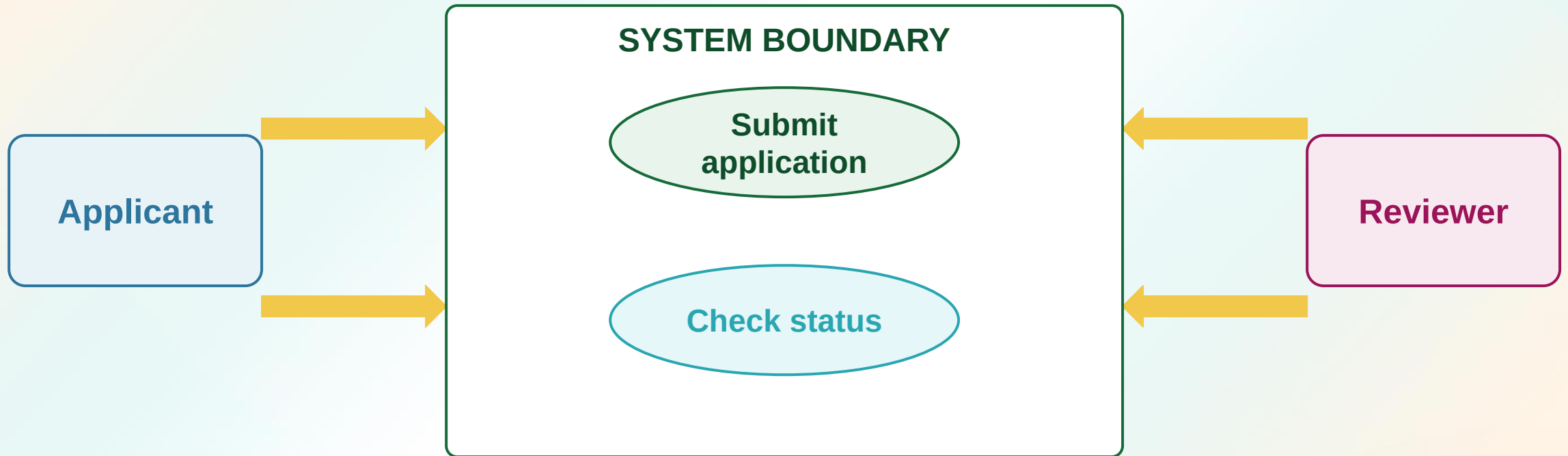
Start with actor goals and a defensible system boundary

An actor is a role outside the system that interacts to achieve a goal—not a named person, screen, database, or internal function.

Actor	Goal	Boundary check
Applicant	Submit evidence and receive status	Human remains external
Reviewer	Evaluate and record a decision	Decision authority is external
Credential provider	Return credential status	External system dependency
Support officer	Resolve failed submissions	Role interacts only when needed

A use case diagram shows goals inside the system boundary

Connect actors to meaningful system services; keep step-by-step behavior in the narrative rather than crowding the diagram.



Use case names are verb phrases that express stakeholder goals or valuable system services.

Use «include», «extend», and generalization only when their meaning fits

These relationships are semantic tools—not decorative arrows or substitutes for ordinary actor association.

Relationship	Meaning	Example
«include»	Required reusable behavior	Submit application includes Validate required fields
«extend»	Optional or conditional behavior	Resolve mismatch extends Review application
Generalization	Specialized actor or use case	Senior reviewer specializes Reviewer
Association	Actor participates in a use case	Applicant — Submit application

A use case narrative explains the complete interaction

Document trigger, actors, preconditions, main success scenario, alternatives, exceptions, postconditions, rules, and linked requirements.

Narrative field	UC-04 Review application
Trigger	A complete application enters the review queue
Precondition	Reviewer is assigned and authorized
Main success	Inspect evidence → record decision → notify applicant
Alternative	Return incomplete evidence with reason
Postcondition	Decision and history are recorded



Strong use cases cover success, alternatives, and failure

Focus on actor intent and observable system behavior; avoid screen-by-screen instructions.

Quality check	Weak narrative	Better direction
Goal focus	Click Review button	Reviewer evaluates an assigned application
Alternatives	Happy path only	Incomplete, mismatch, and unavailable-provider paths
Authorization	Assumed	Precondition and failure response stated
Traceability	No identifiers	Requirements, rules, and acceptance linked

An activity diagram shows workflow and branching behavior

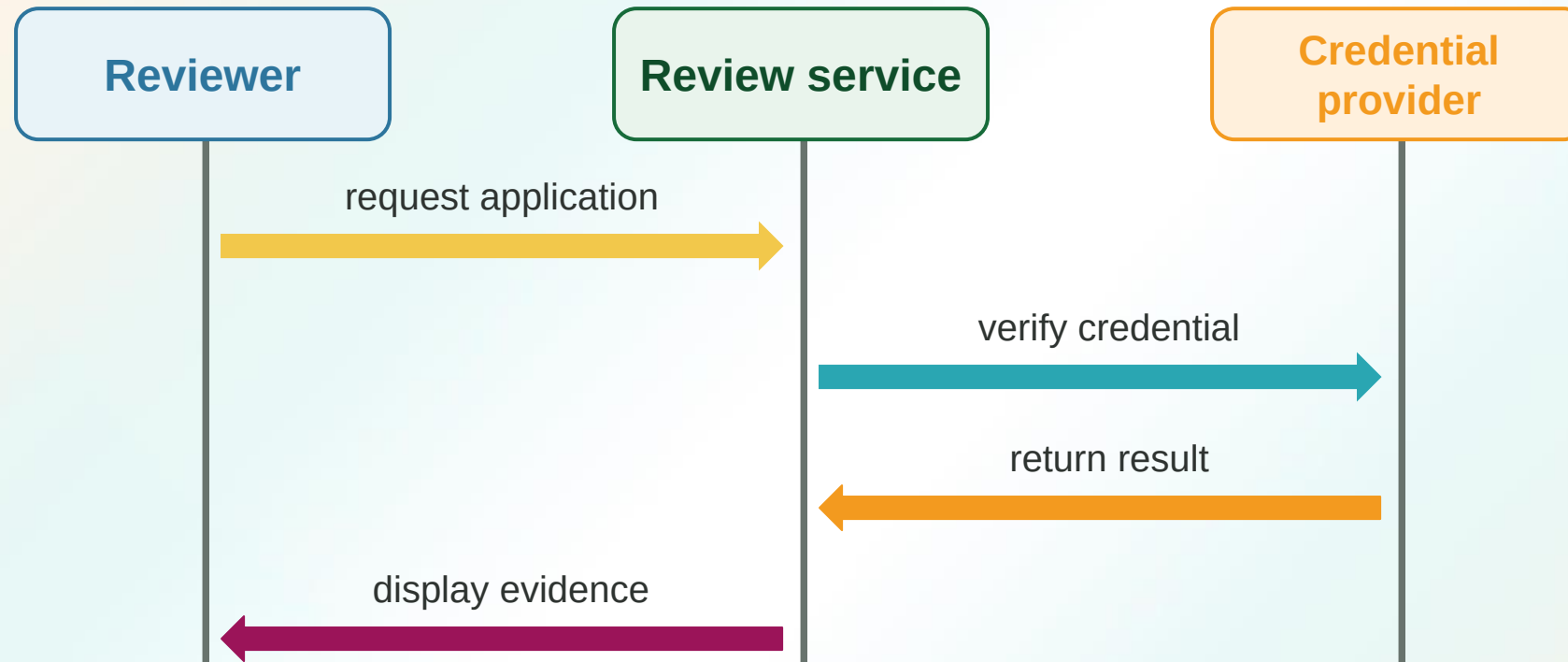
Use actions, decisions, merges, parallel paths, and swimlanes when responsibility or concurrency matters.



Guard conditions should be mutually understandable and traceable to rules: [complete] / [incomplete], not vague labels such as [yes] / [no].

A sequence diagram shows time-ordered interaction

Each participant has a lifeline; messages reveal responsibility, request-response order, and important alternatives.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

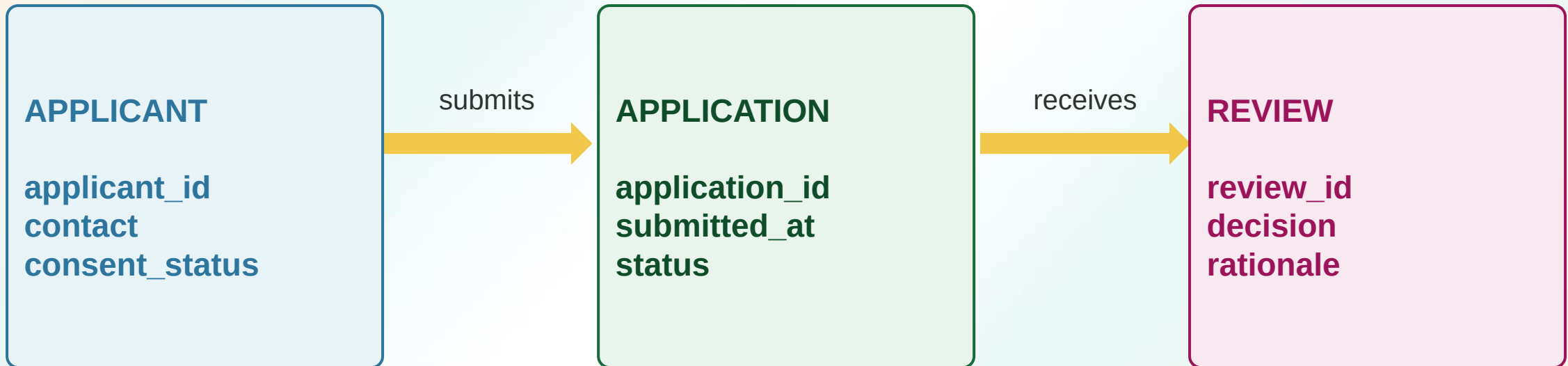
Activity and sequence views must describe the same scenario

The activity diagram explains workflow; the sequence diagram assigns interactions. Compare actions, branches, participants, and outcomes.

Scenario element	Activity view	Sequence view	Consistent?
Request review	Receive assignment	Reviewer → service	Yes
Verify identity	Check credential	Service → provider	Yes
Provider unavailable	Pending branch	Missing alt fragment	No
Record decision	Final action	Service → repository	Check model

Domain models define concepts and relationships

Begin with analysis concepts from the requirements and dictionary; add attributes and responsibilities only when evidence supports them.



Do not turn every noun in the requirements into a class. Model concepts with identity, meaning, behavior, or persistent relationships.

Multiplicity states how many instances may participate

Read both ends of an association and validate optionality, minimums, maximums, and lifecycle conditions with stakeholders and rules.

Association	Multiplicity	Interpretation
Applicant—Application	1 to 0..*	One applicant may submit zero or many applications
Application—Review	1 to 0..*	An application may undergo several review cycles
Review—Reviewer	0..* to 1	Each review is assigned to one reviewer
Application—Credential	1 to 1..*	At least one credential is required

Use an ERD when persistent data structure is the main question

Define entities, identifiers, attributes, relationships, cardinality, and optionality—then reconcile them with the dictionary and DFD stores.



An ERD is not automatically a database design. At analysis level, focus on business meaning and required relationships.

Use state modeling when behavior depends on lifecycle state

A state diagram is valuable for games, workflows, devices, approvals, security controls, and any object with governed transitions.



Each transition needs an event, guard, or rule. Invalid transitions—such as Draft directly to Verified—should be prevented and tested.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Choose model views by the question you must answer

Not every project needs every possible diagram; every selected view must add information and remain consistent with the rest.

Question	Preferred view	Evidence of value
What concepts and responsibilities exist?	Domain / class model	Requirements use stable domain concepts
What persistent data relationships exist?	ERD	Stores and reports depend on cardinality
How does an object change over time?	State model	Rules depend on lifecycle state
How do participants interact?	Sequence diagram	Responsibility and order are uncertain

Cross-model consistency is a deliberate inspection

Compare identifiers, names, scope, actors, actions, data, rules, states, and outcomes across every requirement and model view.

Anchor	Must agree with	Defect signal
Requirement R-04	Use case, DFD process, acceptance	Required behavior missing
DFD flow	Dictionary structure, sequence message	Different meaning or contents
Business rule	P-SPEC, decision table, state transition	Contradictory outcome
Domain concept	Dictionary, ERD/store, use case narrative	Duplicate or undefined term

A model walkthrough turns diagrams into testable explanations

Walk through normal, alternative, failure, and unauthorized scenarios while peers trace each step across models and requirements.



Quality gate: correct notation, complete scope, balanced flows, defined data, explicit logic, consistent UML, and preserved traceability.

AI may critique models—but the team must validate every suggestion

Class Activity 4 uses only synthetic or de-identified model descriptions or diagrams and requires a documented validation record.



Adaptive practice is assigned only when diagnostic or formative evidence shows a need. It is AI-Integrated, ungraded, and not a fixed course deliverable.

Module 4 activity path

Formative Assessment 4 — AI-Prohibited, ungraded: Complete the Canvas quiz after Module 4 independently to check current mastery.

Class Activity 4 — AI-Integrated: Submit the AI Model Consistency Audit with sanitized inputs, prompts and outputs, critique, validation, accepted and rejected advice, corrected models, and AI Use Disclosure.

Summative Assessment 2 — AI-Prohibited, graded: Complete the Canvas quiz after Module 4. Coverage is Modules 3–4.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Module 4 portfolio checkpoint

Structured model set: Context diagram, balanced DFD levels, defined data flows and stores, data dictionary, and process specifications.

Object-oriented model set: Use case diagram and narratives, activity and sequence views, and a domain/class model or ERD; include state modeling when justified.

Consistency evidence: Defect log, peer walkthrough record, corrected models, and RTM links from requirements to model identifiers.

Quality gate: Resolve material inconsistencies before Module 5 prototyping so the prototype does not validate the wrong behavior.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

REFERENCES

- Kendall, K. E., & Kendall, J. E. (2024). Systems analysis and design (11th ed.). Pearson.
- Tilley, S. (2025). Systems analysis and design (13th ed.). Cengage.
- Valacich, J. S., & George, J. F. (2024). Modern systems analysis and design (10th ed.). Pearson.
- Project Management Institute. (2024). Business analysis for practitioners: A practice guide (2nd ed.).
- ISO/IEC. (2023). ISO/IEC 25010:2023 — Product quality model.
<https://www.iso.org/standard/78176.html>
- Object Management Group. (2017). Unified Modeling Language, Version 2.5.1.
<https://www.omg.org/spec/UML/2.5.1>
- Instructor-provided textbook chapter guide, case analysis, specialization case pack, portfolio template, rubric, and peer-review checklist released in Canvas.
- All worked examples in this deck are synthetic and de-identified for instructional use.

**MAKE THE SYSTEM CONSISTENT.
THEN MAKE IT TANGIBLE.**



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation